Distributed Multi-Model Analytics for E-commerce Data

MongoDB, HBase and Apache Spark | Technical Report | AUCA Big Data Analytics Final Project

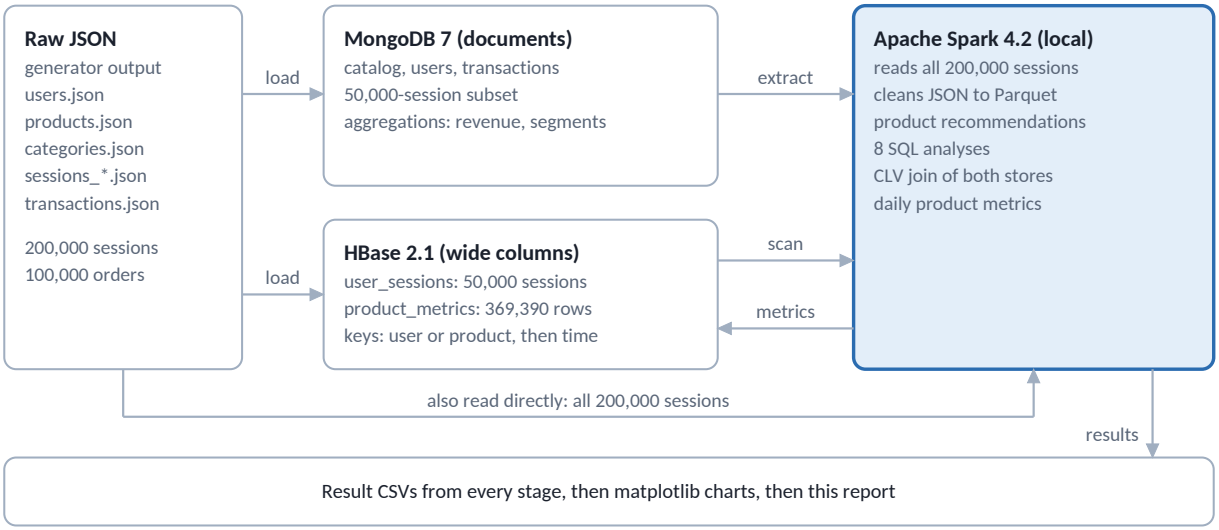
NISHYIRIMBERE Yves | Student ID 101388 | Big Data Analytics, AUCA | 10 November 2026 | Code and results: github.com/Yves-Nishyirimbere/BDAnalytics_FinalExam

200,000	100,000	17.2%	r = 0.08
sessions (about 2.0 million page views)	orders worth $111.1M	of total lifetime value held by the top 10% of buyers	views vs sales among the 75% of products with most views

Summary. A synthetic 90-day e-commerce dataset is split across three engines, each used where its model fits: MongoDB for document-shaped data, HBase for time-ordered session events and product metrics, and Spark for processing everything and joining the two stores. Revenue is stable at about $1.2M a day and spread evenly across 25 categories. Customers differ in how often they buy, not in how much they browse. The apparent link between product views and sales (r = 0.61) comes from a low-view tail of products that is consistent with stock-outs; among the other 75% of products it is negligible (r = 0.08).

1. System architecture

The generator output is loaded into MongoDB and HBase. Spark reads the raw files directly, extracts from both stores for the integrated analysis, and writes daily product metrics that are loaded into HBase.



System architecture: three engines, one integration point (Spark).

Data. Synthetic, from the provided generator, scaled to 10,000 users, 5,000 products in 25 categories, 200,000 sessions and 100,000 transactions over about 90 days (August to November 2026). **Setup.** MongoDB 7 and HBase 2.1.3 run in Docker; Spark 4.2.0 runs in local mode on a MacBook; HBase is reached through its Thrift server with happybase. Only 50,000 of the 200,000 sessions are loaded into each store (faster loading); Spark always sees the full data.

2. Data modeling: MongoDB and HBase

MongoDB holds data that is read as whole documents and aggregated along many dimensions; HBase holds data that is read by key and time range.

MongoDB schema

Collection	Design	Reason
products	Category name and subcategory (id, name, margin) embedded; price_history array embedded	One read per product page; the history is only read with the product
users	Profile plus embedded purchase_summary (orders, spend, average order value, first and last purchase), written by a $merge pipeline	Profile reads need no scan of transactions
transactions	Line items embedded; each item also carries category_id and subcategory_id copied from products	An order is read with its items; revenue by category needs no join
sessions (50,000 subset)	page_views and cart_contents embedded in the session document	One read returns the whole user journey

Indexes follow the query patterns: transactions on user_id, timestamp and items.product_id; products on category_id; sessions on (user_id, start_time) and conversion_status. **Five aggregation pipelines** run on this model: (A1) revenue by category and month with $unwind and $group; (A2) customer segmentation by order count with $bucketAuto, which keeps segments balanced at any data size; (A3) top-selling products from transactions against most-viewed products from sessions; (A4) revenue by US state with $lookup into users; (A5) the $merge pipeline that writes each user's purchase summary.

```
db.transactions.aggregate([
  {$unwind: "$items"},
  {$group: {_id: {category_id: "$items.category_id", month: {$dateToString: {format: "%Y-%m", date: "$timestamp"}}},
            revenue: {$sum: "$items.subtotal"}, units: {$sum: "$items.quantity"}}},
  {$lookup: {from: "categories", localField: "_id.category_id", foreignField: "category_id", as: "cat"}},
  {$sort: {"_id.month": 1, revenue: -1}} ])
```

Pipeline A1 (abridged). All five pipelines are in mongodb/aggregations.py.

HBase schema

Table	Row key	Column families	Query served
user_sessions	user_id#reverse_ts#session_id (reverse_ts = 9999999999999 minus epoch ms)	info (one value per session); pv (sparse page-view columns pv_000, pv_001, ...); cart (sparse, qualifier = product_id)	Latest N sessions of a user; sessions in a time window
product_metrics	product_id#yyyymmdd	m: views, cart_adds, purchases, revenue	Daily history of one product

```
create 'user_sessions', {NAME=>'info', VERSIONS=>1}, {NAME=>'pv', VERSIONS=>1}, {NAME=>'cart', VERSIONS=>1}
create 'product_metrics', {NAME=>'m', VERSIONS=>1}
scan 'user_sessions', {ROWPREFIXFILTER=>'user_000000#', LIMIT=>3, COLUMNS=>['info']}
```

HBase shell commands for the schema and a per-user query (the loaders use happybase; see hbase/hbase_shell_commands.txt).

The reverse timestamp stores each user's sessions newest first, so "latest N sessions" is a short prefix scan and a time window is a start/stop-row range scan. The key leads with user_id, not time, so writes spread across regions instead of piling onto the newest rows. There are only three column families because each is stored separately: queries that need just the summary read *info* and skip the wide *pv* family. **Measured:** 50,000 sessions loaded in 15.3 s and 369,390 metric rows in 11.9 s; in single runs a user's latest five sessions took 22.0 ms, a 30-day window 17.6 ms, and a product's 78-day history 32.1 ms.

Why each dataset sits where it does

Data	Store	Reason
Catalog, users, transactions	MongoDB	Nested documents aggregated by many fields (category, month, user, state)
Session event stream	HBase	High volume, append-only, always fetched by user and time
Daily product metrics	HBase	Fixed-shape counters read by product and date range; bulk-written by Spark
Session subset with full journeys	MongoDB	A complete journey is analysed as one document (for example most-viewed products)

3. Spark processing pipelines

Cleaning and normalisation

Every JSON file is read with explicit schemas: *cart_contents* is keyed by product id, so inference would create a struct with thousands of fields; it is modelled as a map instead. The job parses timestamps, removes duplicate keys, fills missing values, standardises text and flags problems instead of dropping rows. Nested data becomes flat fact tables: 1,997,108 page views, 207,679 cart items and 226,936 transaction items. Products have no subcategory in the generated data, so one is assigned from an MD5 hash of the product id, with the same rule in Python (MongoDB) and Spark. Output is Parquet partitioned by date.

Data-quality check	Result	Data-quality check	Result
Sessions and transactions in vs out	200,000 and 100,000; none lost	Converted sessions with no transaction	0
Transactions with no session link	59,512 (59.5%)	Sessions abandoned / browsed / converted	89,436 / 70,076 / 40,488
Transactions whose totals do not add up	0	Inactive products	234 of 5,000

Product recommendations

Baskets are transactions ("bought together") or sessions ("viewed together"). A self-join on basket id with product_a < product_b counts each pair once; support, confidence and lift are computed, with a minimum pair count (2 for purchases, 3 for views). The job kept 1,487 bought-together pairs (average lift 82.1, maximum 340.1) and 2,139 viewed-together pairs (average 26.6, maximum 125.8). These lifts are small-count artefacts, not real affinities: in the top pair the two products appear in only 49 and 12 of 100,000 baskets, so two shared baskets give a lift of 340. The generator picks products at random, so the method works but there is no signal for it to find.

Spark SQL analyses

Query	Technique	Query	Technique
Q1 Top 3 categories per month	CTE and RANK() window	Q5 Revenue vs estimated profit by category	Joins to dimension tables
Q2 Conversion by device and referrer	GROUPING SETS	Q6 Strict conversion funnel	Conditional aggregation
Q3 Daily revenue, 7-day moving average	Window frame	Q7 Product views vs units sold	Two aggregates joined to the catalog
Q4 Spend quartiles and engagement	Three-way join and NTILE	Q8 Buyers by country: spend, engagement	MongoDB users joined with HBase sessions

```
WITH stage AS (
  SELECT session_id,
    MAX(CASE WHEN page_type='product_detail' THEN 1 ELSE 0 END) AS viewed,
    MAX(CASE WHEN page_type='cart'           THEN 1 ELSE 0 END) AS cart,
    MAX(CASE WHEN page_type='checkout'       THEN 1 ELSE 0 END) AS checkout,
    MAX(CASE WHEN page_type='confirmation'   THEN 1 ELSE 0 END) AS confirmation
  FROM page_views GROUP BY session_id)
SELECT COUNT(*) AS all_sessions, SUM(viewed) AS viewed_product, SUM(viewed*cart) AS added_to_cart,
       SUM(viewed*cart*checkout) AS reached_checkout, SUM(viewed*cart*checkout*confirmation) AS reached_confirmation
FROM stage JOIN sessions s ON stage.session_id = s.session_id
```

Example query (Q6, strict funnel, abridged). All eight queries are in spark/03_sql_analytics.py.

Optimisation choices. Explicit schemas avoid a pass over the JSON to infer types; Parquet is columnar, so later jobs read only the columns they need, and date partitioning allows time-range pruning; reused DataFrames are cached; shuffle partitions are set to 16 for one machine. The raw files are one JSON array each, which Spark cannot split, so read parallelism equals the file count; JSON Lines would remove that limit.

4. Integrated analytics: customer lifetime value

Business question. Which customers are the most valuable going forward, how concentrated is revenue in them, and does browsing engagement go together with spend?

System	Data used
MongoDB	Each user's profile and embedded purchase summary (orders, spend, average order value, country)
HBase	Per-session duration and outcome from user_sessions, for the 50,000 loaded sessions
Spark	The only engine that sees both, so it performs the join and the computation

Steps. (1) Extract users and purchase summaries from MongoDB with pymongo into a Spark DataFrame with an explicit schema. (2) Scan four small columns of user_sessions from HBase with happybase, never the wide page-view family. (3) Aggregate sessions per user: count, average duration, converted and abandoned sessions. (4) Left-join on user_id, so users with no sessions in the HBase subset keep a count of 0. (5) Compute CLV and rank buyers with percent_rank into four tiers.

> $$CLV = AOV \times (orders \div 2.96 \text{ months observed}) \times 12 \text{ months expected lifespan}$$

The 90-day window equals 2.96 months; the 12-month lifespan is an explicit, adjustable assumption. The result ranks customers and is not a revenue forecast.

Tier	Customers	Avg CLV ($)	Avg orders	Avg sessions (HBase)	Share of total CLV
Platinum (top 10%)	1,000	77,520	15.0	5.3	17.2%
Gold (next 20%)	2,000	59,037	12.2	5.2	26.2%
Silver (next 30%)	2,999	46,065	10.3	5.1	30.7%
Bronze (bottom 40%)	3,999	29,156	7.4	4.8	25.9%

Value is not concentrated: the top 10% of buyers hold 17.2% of total CLV and the bottom 40% hold 25.9%. Engagement is nearly unrelated to value: session count correlates with total spend at 0.092, and average session length with CLV at -0.029. Session counts here (about 5 per user) come from the 50,000-session HBase subset; the full Spark data averages 19 to 21 per user. **Performance:** only four small HBase columns are scanned in large batches, the MongoDB extract projects only needed fields, and the join is against a 10,000-row user table. The global percent_rank window runs in one partition, which is fine for 10,000 buyers; at scale it would be partitioned or replaced by approximate quantiles.

5. Key findings and visualizations

Revenue is steady at about $1.2M a day, customers differ mainly in how often they buy, and conversion, engagement and product popularity show little structure beyond stock availability. Every chart is produced from the result files written by the MongoDB, Spark and integration stages.

Sales performance (Figure 1). Daily revenue holds between about $1.2M and $1.3M (median $1.23M), totalling $111.1M over 100,000 orders (average order about $1,111). The 25 categories each earn $4.0M to $5.5M and the top 10 account for 43.1% of

item revenue. Kelly-Santiago leads with $5.45M and ranks first in August, September and October, yet by estimated profit it is only 16th of 25 (20.4% margin); Rodriguez-Graham earns the most ($1.61M at a 36.2% margin). The overall estimated margin is 25.2%, but the margins rest on the assigned subcategories and illustrate the method more than real profit.

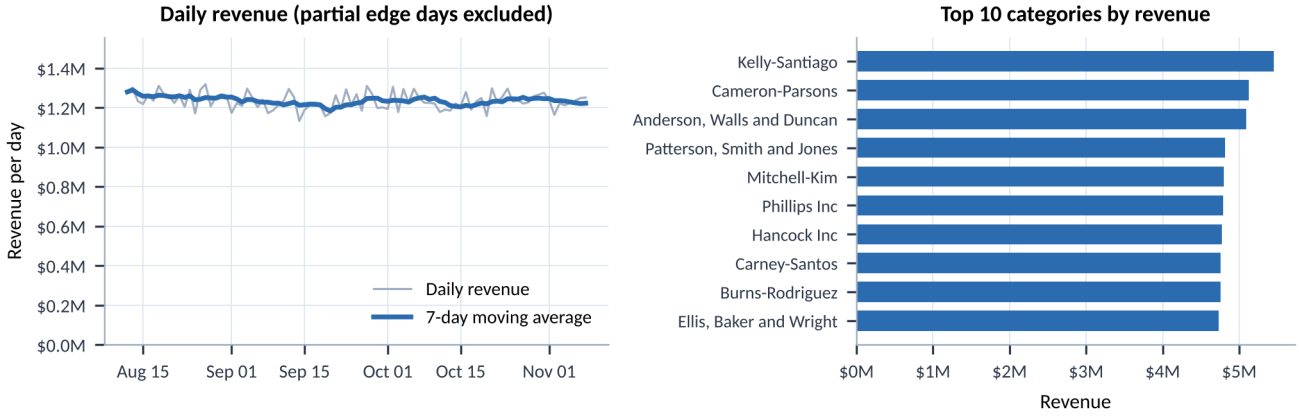


Figure 1. Daily revenue with 7-day average (partial first and last days excluded) and the ten largest categories by revenue.

Customer segmentation (Figure 2). Buyers fall into five near-equal groups by order count, and average spend rises from $6,561 (1 to 7 orders) to $18,240 (15 to 23 orders). Revenue is not concentrated in a few customers (top 10% by CLV: 17.2% of the total). Engagement barely changes with spend: across spend quartiles, average sessions per user move only from 19.0 to 21.1 and session length stays near 1,810 seconds.

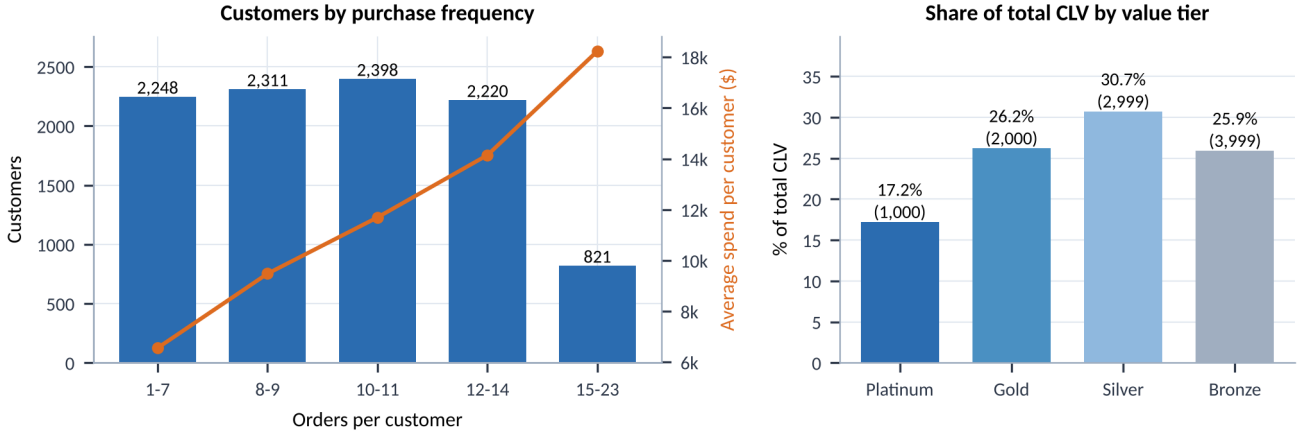


Figure 2. Customers by purchase frequency with average spend (left) and share of total CLV by value tier (right).

Product performance (Figure 3). Across all active products, views and units sold correlate at r = 0.61, but among the 75% of products with the most views the correlation is only r = 0.08. The overall figure comes from a tail of low-view, low-sales products: in the bottom 10% by views (460 products), 53.9% have zero stock left, against 3.0% of the others, and average remaining stock is 204 units against 439. Product age does not explain it (the latest creation date in both groups is June 12, before the data window). This is consistent with stock-outs driving the relationship, though it shows association, not proof. The 234 inactive products have no views or sales and are not plotted.

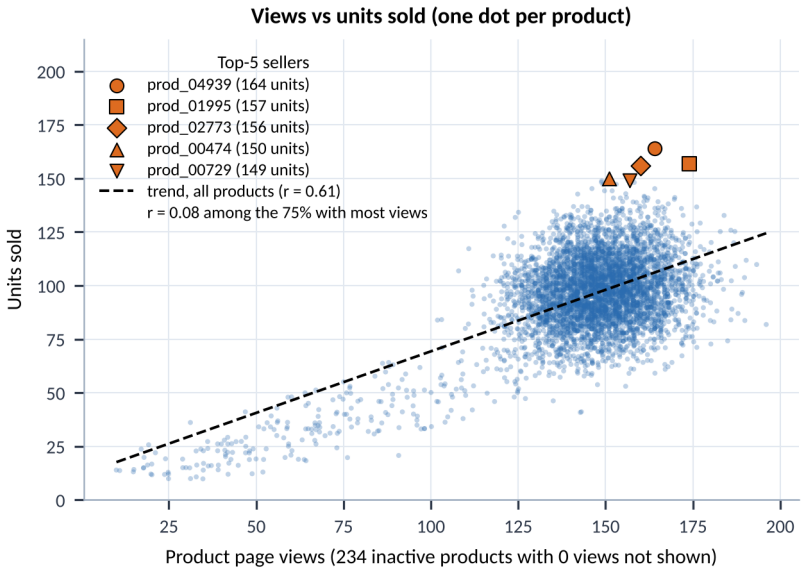


Figure 3. Views vs units sold, one dot per product.

Conversion funnel (Figure 4). 68% of sessions add to cart, 44% reach checkout and 33% reach the confirmation page. Among page-to-page steps, cart to checkout loses the most (64% continue). The strict path ends with 30,772 purchasers (15%), yet 40,488 sessions (20.2%) are marked converted and each has a transaction, so the page-view path undercounts conversions by about a quarter.

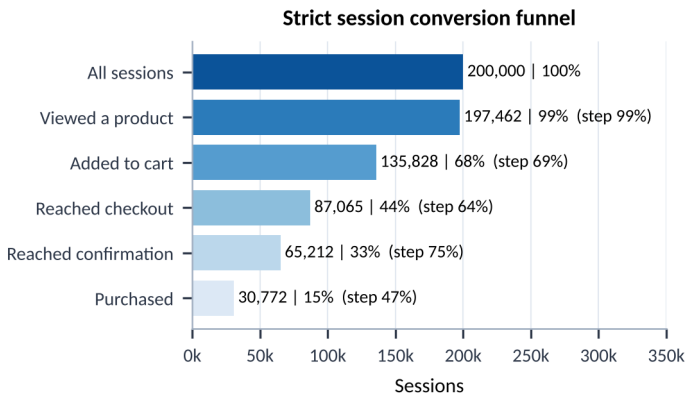


Figure 4. Strict session conversion funnel.

Conversion by device and referrer (Figure 5). Conversion is 20.2% to 20.3% across devices and 19.9% to 20.6% across referrers. No channel stands out, so in this data neither device mix nor traffic source is a lever for conversion.

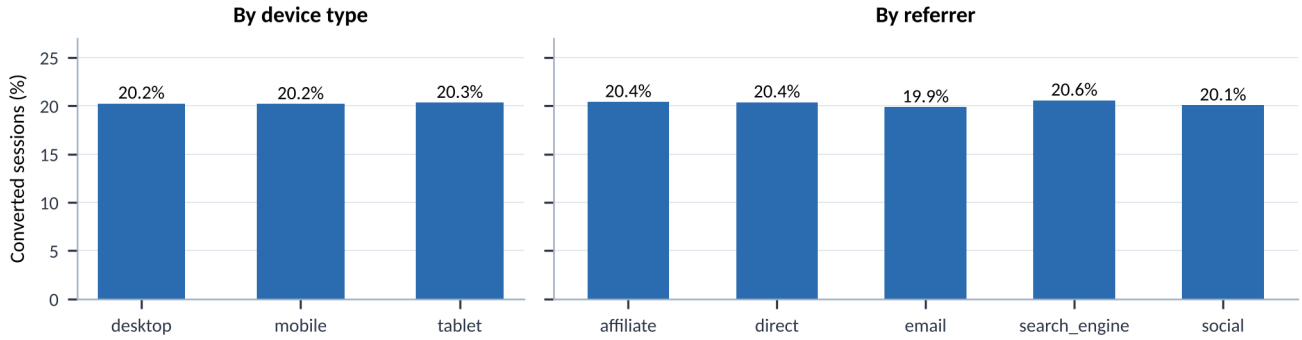


Figure 5. Share of converted sessions by device type and by referrer.

What this means for the business

- **Protect availability.** 53.9% of the lowest-view products are already out of stock, against 3.0% of the rest. Restocking the sold-out tail is the most direct way to recover sales that browsing does not explain (an association to confirm with stock history).
- **Optimise for margin, not only revenue.** The top-revenue category ranks 16th of 25 by estimated profit, while Rodriguez-Graham earns the most. Promoting high-margin categories raises profit without more traffic (margins are illustrative).
- **Fix the cart-to-checkout step.** It loses the most sessions of any page-to-page step (64% continue), so it is the largest controllable drop in the funnel.
- **Grow repeat purchases across the base.** Spend rises from $6,561 to $18,240 as orders go from 1-7 to 15-23, and the top 10% of buyers hold only 17.2% of CLV: value comes from many repeat customers, so loyalty offers should reach the broad base.
- **Do not prioritise by channel.** Conversion is 19.9% to 20.6% for every device and referrer.

Method notes. Daily revenue uses order totals after discount ($111.1M); category revenue and shares use item subtotals before discount. Correlations are Pearson; the "75% with most views" group is products at or above the 25th percentile of views. The funnel is strict: a session counts at a stage only if it also reached every earlier stage. Estimated profit is item revenue times the margin of the assigned subcategory. Charts are drawn from the result CSVs written by the pipeline (viz/make_charts.py in the repository, redrawn at print size for this report).

6. Scalability considerations

Everything here ran on one laptop (Docker stores, local Spark), so the points below describe expected scaling and were not measured at larger size.

Component	How it scales
MongoDB	Shard transactions and sessions on a hashed user_id: one user's documents stay together and load spreads evenly. Embedded items avoid cross-shard joins, the denormalised category keeps revenue pipelines on one collection, and the $merge summary can be refreshed incrementally.
HBase	Keys lead with user_id or product_id, so writes spread across regions instead of hot-spotting on new timestamps. Tables would be pre-split by key prefix (example splits are in hbase/hbase_shell_commands.txt) with region servers added as volume grows; empty sparse cells cost nothing.
Spark	The same code runs on a cluster. Date-partitioned Parquet lets time-range queries read only needed files; small dimension tables would be broadcast-joined; the co-occurrence self-join needs the minimum-count filter and skew handling for very popular products; ingest should move from one-array JSON to JSON Lines or Parquet so reads can be split.
Integration	The CLV job extracts through single Python clients, which is fine at 50,000 sessions. At production scale HBase data would be aggregated inside the cluster and the Spark connectors for MongoDB and HBase would replace client-side extraction.

7. Technology selection by analytical task

Task	Technology	Why this one
Revenue by category and month, segmentation, revenue by state	MongoDB aggregation	Orders are documents with embedded items; pipelines group by many fields without joins
Latest sessions of a user; sessions in a time window	HBase	The reverse-timestamp key turns both into short range scans (17.6 to 22.0 ms measured)
Daily metrics of one product	HBase	Keys are product and date, so one range scan returns the history
Cleaning and reshaping 200,000 sessions (about 2.0M page views)	Spark	Distributed reads and writes, explicit schemas, columnar output
Co-occurrence recommendations	Spark	A self-join and aggregation over millions of rows is a distributed job
Funnel, window functions, cross-table SQL	Spark SQL	Declarative, readable, auditable queries over Parquet
Lifetime value across profile, orders and engagement	Spark joining MongoDB and HBase extracts	Neither store holds both inputs; only Spark can combine them and compute over the result

8. Limitations and future work

Limitations

- **Random data.** Co-purchase lifts are chance-level, engagement is flat across spend groups, and conversion barely differs by device or referrer (19.9% to 20.6%).
- **Store subsets.** MongoDB and HBase hold 50,000 of 200,000 sessions, so engagement in the CLV and cross-store analyses reflects a quarter of the sessions.
- **Assigned subcategories.** Margins come from a hash-based assignment, so profit by category is illustrative.
- **No demographics.** Segments use order frequency, spend and CLV tier.
- **Funnel.** The strict page path counts 30,772 purchasers against 40,488 converted sessions, so it undercounts by about a quarter.
- **Simple CLV.** It extrapolates 90 days to 12 months with no churn model; amounts (about $11,000 per buyer) suit ranking only.
- **Loose links.** 59.5% of transactions have no session; dates are relative to generation time and the first and last days are partial.
- **Single node.** Docker HBase and local Spark show the design, not cluster behaviour; extraction uses client libraries, not Spark connectors.

Future work

- Spark connectors for MongoDB and HBase, and a simulated stream (Structured Streaming) for near-real-time funnel metrics.
- Collaborative filtering (ALS) on data with real preference structure.
- A probabilistic CLV model that handles churn.
- Stock levels tracked over time, to test whether stock-outs explain the low-view, low-sales product tail.
- A multi-node re-run to measure scaling.

9. Reproducibility and repository

The repository (github.com/Yves-Nishyirimbere/BDAnalytics_FinalExam) holds all code, the result files, the charts and screenshots of each stage running. To reproduce the results: generate the data with data_generation/prepare_generator.py; then run mongodb/load_mongo.py and mongodb/aggregations.py; spark/01_clean.py and spark/04_product_metrics.py; hbase/create_tables.py, hbase/load_hbase.py, hbase/load_product_metrics.py and hbase/queries.py; spark/02_recommend.py and spark/03_sql_analytics.py; integration/clv_pipeline.py; and viz/make_charts.py. The script run_all.sh runs the same sequence, and the README lists prerequisites and settings. Dates in the generated data are relative to the day it is generated, so absolute dates (and the partial first and last days) shift if the data is regenerated.